

AI Document Processing System

Architecture Design Document

Automated Document Analysis, Drafting & Review using RAG + LLMs

Core Technology Stack			
Frontend Streamlit UI	Backend API FastAPI (Python)	LLM & Embed Groq	Orchestration LangChain
Vector DB FAISS	Web Search Tavily	Database SQLite	

Executive Summary

This document provides a comprehensive architectural design rationale for the **AI Document Processing System** — a multi-agent pipeline that automates document ingestion, analysis, intelligent drafting, and human-in-the-loop review. The system orchestrates Retrieval-Augmented Generation (RAG) with live web search to produce contextually accurate, up-to-date document drafts, while ensuring a mandatory human approval gate before any output is finalized.

Beyond the architecture justification, this document covers **market research** situating the system within the broader Document AI landscape, an **evaluation strategy** for continuously measuring LLM and retrieval quality, and a **logging & observability design** for production reliability.

Design Goal	How the Architecture Addresses It
Accuracy	RAG grounding + web search minimise hallucination; human review catches residual errors
Auditability	Full audit trail in SQLite; every approval is timestamped and attributed
Extensibility	Modular LangChain agents allow swapping retrieval, search, or LLM backends independently
Observability	Structured logging, LangSmith tracing, and evaluation metrics expose system health
Cost-efficiency	Groq + SQLite + FAISS minimise managed-service spend

1. System Overview & Pipeline

1.1 Pipeline Stages

The system follows a seven-stage sequential pipeline with parallel enrichment paths converging at a unified draft generation stage:

Stage	Name	Responsibility
1	User Uploads Document	Accepts PDF, DOCX, or TXT via Streamlit UI
2	Extract & Preprocess	Parses, cleans, and chunks document text
3	Agent 1 — RAG Retrieval	Embeds chunks, stores in FAISS, retrieves relevant context
4	Agent 2 — Live Web Search	Fetches real-time web insights via SerpAPI / Tavily
5	Draft Generator (Agent)	Synthesises retrieved context + web insights into a draft
6	Human Review	Editor reviews, annotates, and approves or requests changes
7	Final Output	Approved draft persisted in SQLite and exported via FastAPI

2. Technology Stack — Design Justifications

2.1 Frontend UI — Streamlit

Streamlit was chosen for its rapid prototyping capability and Python-native integration, eliminating a dedicated frontend engineering effort. It provides file-upload widgets, real-time status display, and an editor panel out of the box.

Pros	Cons
Zero-config Python UI — no JavaScript required	Limited customisation vs React / Vue
Built-in file upload with progress indicators	Single-threaded execution bottlenecks concurrent users
Rapid iteration: UI changes reflected immediately	Not suitable for high-traffic public applications
Native integration with LangChain and pandas	State management can be complex at scale
Ideal for internal tools and data-heavy workflows	Mobile responsiveness is limited

Alternatives Considered

Alternative	Why Not Chosen
React + Next.js	Requires dedicated frontend developer; overkill for internal tooling

Gradio	Narrower widget library; less control over layout and review UI
Flask + Jinja2	Full templating overhead; slower iteration for data applications

2.2 Backend API — FastAPI (Python)

FastAPI serves as the system backbone exposing REST endpoints for upload, review, and export. Its async-first design pairs naturally with I/O-heavy operations such as embedding generation and vector DB queries, avoiding blocking calls that degrade throughput.

Pros	Cons
Async support via asyncio — non-blocking LLM calls	Younger ecosystem vs Django — fewer mature plugins
Automatic OpenAPI/Swagger documentation	No built-in ORM; requires SQLAlchemy or raw SQL
Pydantic request validation with minimal boilerplate	Mixed sync/async code can complicate debugging
Python-native — shares type system with LangChain	Less opinionated structure risks inconsistency at scale
High throughput for I/O-bound workloads	

Alternatives Considered

Alternative	Why Not Chosen
Django REST Framework	Sync-first model adds latency for concurrent LLM API calls
Flask	Lacks native async support; additional wiring required
Node.js (Express)	Context-switch from Python ML stack; loses type parity

2.3 LLM & Embeddings — Groq / Llama 3

Groq provides inference for both text generation (Llama 3) and embedding creation via its hardware-accelerated LPU architecture, delivering significantly lower latency than GPU-based alternatives — critical in a feedback-loop design where multiple LLM calls occur per cycle.

Pros	Cons
Industry-leading inference speed via LPU hardware	Cloud availability subject to provider reliability
Llama 3 is open-weights — auditable and fine-tunable	Llama 3 context window smaller than GPT-4 Turbo
Single provider for LLM inference and embeddings	Limited native function-calling support
Cost-effective vs GPT-4 / Claude at high volume	Embedding models less proven than text-embedding-3-large
Compatible with OpenAI SDK interface	Vendor lock-in risk if LPU API surface changes

Alternatives Considered

Alternative	Why Not Chosen
-------------	----------------

OpenAI GPT-4o	Higher cost at scale; closed weights limit auditability
Anthropic Claude API	Excellent quality; higher cost and less LangChain embedding support

2.4 Orchestration — LangChain

LangChain orchestrates agent workflows, tool integrations (RAG + web search), prompt templating, and the multi-step reasoning chain connecting retrieval to generation. LangSmith provides native tracing for debugging agent runs.

Pros	Cons
Mature agent framework with pre-built RAG chains	Abstraction overhead obscures debugging in complex chains
Native FAISS and vector DB integrations	Rapidly evolving API — version upgrades can be breaking
Modular tool abstractions — easy to swap backends	Performance overhead vs hand-rolled pipelines
LangSmith observability for tracing agent runs	Heavy dependency footprint

Alternatives Considered

Alternative	Why Not Chosen
LlamaIndex	Better for pure RAG/indexing; weaker multi-agent orchestration
Haystack (deepset)	Strong NLP pipeline; smaller LLM agent ecosystem
AutoGen (Microsoft)	Multi-agent strengths but heavier setup; less RAG-native

2.5 Vector Database — Pinecone

Pinecone provides a fully managed cloud-native vector database for semantic search, enabling scalable, low-latency retrieval with built-in metadata filtering, automatic indexing, and high availability without managing infrastructure.

Pros	Cons
Fully managed service — no infrastructure to maintain	Higher operational cost than self-hosted solutions
Built-in metadata filtering and namespace support	External service dependency requiring network access
Automatically scales horizontally with growing datasets	Vendor lock-in compared to open-source alternatives
High availability, backups, and production-grade reliability	Requires internet connectivity for retrieval operations
Native integrations with LangChain	Less flexibility over index internals and tuning

Alternatives Considered

Alternative	Tradeoff vs FAISS
FAISS	Lacks managed scalability, metadata filtering, and persistence by default

Weaviate	Rich hybrid search (vector + BM25); requires Docker/K8s
Chroma	LangChain-native dev setup; less proven at large scale

2.6 Live Web Search — Tavily

Agent 2 augments static document knowledge with real-time web retrieval, ensuring drafts reflect current information. Tavily is purpose-built for LLM workflows with structured snippet extraction; SerpAPI provides Google Search result coverage.

Pros	Cons
Tavily returns LLM-optimised structured snippets	API costs scale linearly with document volume
SerpAPI covers multiple search engines for breadth	Rate limits can bottleneck high-throughput pipelines
Dual-provider redundancy if one API is unavailable	Search quality varies by domain
LangChain tool wrappers available for both	Adds 500ms–2s latency per retrieval cycle
Bridges Llama 3 knowledge cutoff gap	Noisy results risk polluting draft context

Alternatives Considered

Alternative	Why Not Chosen
Bing Search API	Good quality but requires Azure account; less LangChain-native
DuckDuckGo API	Free but rate-limited; unreliable for production workloads
Perplexity API	AI-synthesised results useful but less controllable for grounding

2.7 Relational Database — SQLite

SQLite stores users, documents, drafts, reviews, approvals, and audit logs. Its serverless, file-based architecture eliminates database infrastructure overhead during the initial deployment phase while providing full ACID compliance.

Pros	Cons
Zero-config serverless deployment — no DB server process	Single writer at a time — limited write concurrency
ACID-compliant with WAL mode for concurrent reads	Not suitable for distributed or multi-node deployments
Single-file backup — trivial disaster recovery	Limited data types vs PostgreSQL
Python sqlite3 in standard library — zero dependencies	Migration tooling less mature than Alembic + PostgreSQL
Sufficient for thousands of documents/day without tuning	No native row-level security

Alternatives Considered

Alternative	When to Migrate
PostgreSQL	When concurrent writes exceed SQLite thresholds (>100/min)
MySQL / MariaDB	Viable alternative; stronger replication and HA support
MongoDB	When document schemas become highly variable; schema-on-read preferred

3. Market Research & Competitive Landscape

3.1 Document AI Market Overview

The Document AI and Intelligent Document Processing (IDP) market has experienced exponential growth driven by enterprise demand for automated knowledge extraction and generative drafting. Key market signals informing this architecture.

3.2 Competitive Platform Analysis

The following platforms represent the competitive landscape this system competes with or differentiates from:

Platform	Strength	Weakness	Our Differentiation
Microsoft Copilot for M365	Deep Office integration	Closed, Azure-locked, expensive	Open-weight, auditable, extensible
Google Vertex AI Search	Scalable managed RAG	GCP lock-in, limited custom agents	Full pipeline control, on-prem capable
AWS Textract + Bedrock	Strong OCR + multi-model	Complex pricing, fragmented tooling	Unified Python-native stack
Notion AI	Consumer-friendly UX	No RAG, no agent pipeline	Structured multi-agent retrieval
DocuSign Intelligent Agreement	Legal workflow native	Domain-specific, not general-purpose	General-purpose with pluggable domains

3.3 Market Positioning

Dimension	Position
Target Segment	Mid-market enterprises with regulated document workflows (legal, compliance, HR)
Deployment Model	On-premise or private cloud; no data leaves customer environment
Pricing Model	Self-hosted open-source core; support/managed service tier optional
Key Differentiator	Full auditability (SQLite trail) + open-weight LLM + modular agent pipeline

Land-and-expand	Start with single department; expand retrieval domains and agent toolsets
-----------------	---

4. Evaluation Strategies & Logging

4.1 LLM Output Evaluation

LLM evaluation must be continuous, automated, and multi-dimensional. The following framework is applied at each draft generation cycle:

Metric	Method	Tooling	Target
Faithfulness	Does the draft only assert facts from the retrieved context?	RAGAS faithfulness scorer	> 0.85
Answer Relevancy	Is the draft responsive to the original document intent?	RAGAS answer_relevancy	> 0.80
Contextual Precision	Are retrieved chunks relevant to the question?	RAGAS context_precision	> 0.75
Hallucination Rate	Rate of claims not grounded in context or web results	LangSmith + custom NLI scorer	< 5%
Human Approval Rate	% of drafts approved without revision request	SQLite review audit log	> 70%

Alternatives — LLM Evaluation Frameworks

Alternative	Tradeoff
DeepEval	Strong assertion-based testing; less RAG-native than RAGAS
TruLens	Excellent LLM tracing; primarily for OpenAI models
Promptfoo	Best for prompt A/B testing; less suited for RAG pipeline metrics

4.2 RAG Retrieval Evaluation

Retrieval quality is evaluated independently from generation quality to isolate failure modes between the vector search and the LLM drafting layers:

Metric	Description	Target
Context Recall	Fraction of ground-truth facts present in retrieved chunks	> 0.80
MRR@K	Mean Reciprocal Rank of the first relevant chunk at K=5	> 0.75
Hit Rate@K	% of queries where at least one relevant chunk is in top K	> 0.85

Chunk Overlap Rate	% of retrieved chunks with high cosine overlap (redundancy)	< 20%
--------------------	---	-------

4.3 System Performance Metrics

Metric	Measurement Point	SLO Target
End-to-end latency (upload → draft)	Stage 1 → Stage 5	< 15 seconds P95
Embedding generation throughput	Stage 3 (RAG Agent)	> 100 chunks/sec
Vector search latency	FAISS similarity search	< 50ms P99
Web search latency	Stage 4 (SerpAPI/Tavily)	< 2s P95
API response time (FastAPI)	All REST endpoints	< 200ms P95
Draft approval time (human)	Stage 6 (human)	< 30 min median
System availability	Full pipeline uptime	> 99.5% monthly

4.4 Logging Architecture

Structured logging is applied at every pipeline stage to enable debugging, compliance auditing, and performance regression detection:

Log Category	Fields Captured	Storage	Retention
Request logs	timestamp, user_id, document_id, file_type, size_bytes	SQLite + file	90 days
Pipeline stage logs	stage_id, duration_ms, tokens_in, tokens_out, status	SQLite	90 days
LLM call logs	model, prompt_hash, completion_tokens, latency_ms, cost_usd	LangSmith	30 days
Retrieval logs	query_embedding_hash, top_k_scores, chunk_ids_returned	SQLite	30 days
Review audit logs	reviewer_id, action (approve/reject/edit), timestamp, diff_hash	SQLite	7 years
Error logs	stage, exception_type, stack_trace, document_id	File + alert	180 days

Logging Design Justification

Pros	Cons
Structured JSON logs are machine-queryable — no regex parsing	High-cardinality logs grow storage quickly — need retention policies

Per-stage duration logging isolates bottlenecks precisely	LangSmith external dependency adds cost and data egress
Prompt hash logging avoids storing PII while enabling deduplication	Stack trace logging may expose internal architecture details
Review audit logs support regulatory compliance (GDPR, SOC 2)	Log volume spikes during peak processing can overwhelm local storage
Cost_usd field enables per-document LLM cost attribution	

4.5 Observability Stack

The observability stack extends structured logging with distributed tracing, metric dashboards, and alerting:

Layer	Chosen Tool	Justification
LLM Tracing	LangSmith	Native LangChain integration; step-level chain tracing
Metrics & Dashboards	Prometheus	Industry-standard; open-source; FastAPI exporter available

Alternatives — Observability

Alternative	Tradeoff vs Chosen Stack
Datadog	Excellent APM but expensive at scale; vendor lock-in
New Relic	Full-stack observability; cost scales with data volume
OpenTelemetry (OTEL)	Vendor-neutral; requires backend (Jaeger/Tempo) — more setup
AWS CloudWatch	Zero-infra on AWS; limited cross-cloud portability

5. Data Flow Architecture

The system maintains three independent storage layers serving distinct purposes:

Storage Layer	Technology	Data Stored
Vector Store	Pinecone	Document chunk embeddings
Relational DB	SQLite	Users, docs, drafts, reviews, approvals, audit logs
File Storage	Local / Cloud	Original uploads, exported final outputs

6. Human-in-the-Loop Design

Stage 6 (Human Review) is a deliberate architectural gate, not an optional step. This decision reflects legal, quality, and governance requirements:

- Accuracy assurance: LLMs can hallucinate; human review catches factual errors before finalisation
- Regulatory compliance: Legal, medical, and financial workflows require human sign-off (EU AI Act, HIPAA)
- Feedback quality: Human annotations become a fine-tuning signal for iterative model improvement
- Auditability: Every approval is timestamped and attributed in SQLite, forming a full audit trail
- Liability: Human approval transfers accountability appropriately for high-stakes document decisions

7. Scalability & Evolution Path

7.1 Phase 1 — Current Architecture

- Single-server deployment with in-process FAISS and SQLite
- Supports approximately 500 documents/day with sub-second vector retrieval
- File storage on local disk with optional cloud sync (S3, GCS)
- Streamlit and FastAPI co-located on a single Python process

7.2 Phase 2 — Scaling Triggers & Upgrades

Scaling Trigger	Recommended Upgrade
Concurrent writes > 50/min	Migrate SQLite → PostgreSQL with connection pooling
Vector index > 10M chunks	Migrate FAISS → Pinecone (managed)
High document volume (>5K/day)	Containerise pipeline stages; deploy to Kubernetes
API key management	Groq, Tavily keys stored in environment variables; never in code or DB

8. Security Considerations

Security Control	Implementation
Authentication	FastAPI OAuth2 middleware (optional in Phase 1; mandatory in Phase 2)
File validation	MIME type checking and size limits on upload to prevent malicious file injection
Data isolation	Each processing session namespaced by user_id and document_id in all storage layers
Audit logging	All review actions written to SQLite with timestamp, user attribution, and diff hash

Prompt injection	Input sanitisation applied before user content is inserted into prompt templates
Rate limiting	FastAPI middleware rate-limits upload and review endpoints per user per hour

9. Key Risks & Mitigations

Risk	Like.	Impact	Mitigation
LLM hallucination in drafts	Med	High	Mandatory HITL review; RAGAS faithfulness gate in CI
Groq API outage	Low	High	Fallback to OpenAI as secondary provider
Web search result poisoning	Low	Med	Confidence scoring; domain allowlist; reviewer sanity check
SQLite write contention	Med	Low	WAL mode enabled; plan PostgreSQL migration at scale threshold
Sensitive document exposure	Low	High	Enforce auth on all API endpoints; encrypt storage at rest
Prompt injection via document	Low	High	Input sanitisation; sandboxed prompt execution with strict templates

10. Conclusion

The AI Document Processing System architecture represents a pragmatic balance between development velocity, operational simplicity, and production readiness. The technology choices — Streamlit, FastAPI, Groq, LangChain, Pinecone, Tavily, and PostgreSQL — are individually well-justified and collectively coherent: they share a Python-native ecosystem, minimise external service dependencies, and enable a small team to build, test, and deploy the full pipeline rapidly.

The market research confirms strong commercial timing: RAG-based document AI is growing at ~30% CAGR and open-weight LLM adoption is accelerating in enterprise. The system's differentiation — full auditability, human-in-the-loop compliance gate, and open-weight model selection — directly addresses the gaps left by closed-source competitors like Microsoft Copilot and Google Vertex AI Search.

The evaluation strategy (Evaluation Metrics + LangSmith) and observability stack (Prometheus) ensure the system can be operated reliably in production with quantitative quality gates at each release. Structured logging with defined retention policies satisfies both operational debugging needs and compliance audit requirements.